

AI Smart Navigation Assistant for Visually Impaired People Using Raspberry Pi

Smart Cane Environmental Awareness System

Final Project Report

Igor Xavier · Fang Jialuo

IoT Term Project



Repository: <https://github.com/wlggor/smart-nav-cane>

June 26, 2026

Abstract

This report presents the design and implementation of a Smart Cane Environmental Awareness System: an offline, edge-AI assistive device for visually impaired users built entirely on a Raspberry Pi 4. The system combines a Time-of-Flight (ToF) depth camera, a USB webcam, and Bluetooth audio to provide three capabilities: (1) real-time obstacle detection and classification, (2) visual place recognition for trained indoor locations, and (3) voice-driven guided navigation with turn-by-turn audio instructions. The project deliberately combines classical computer vision (ORB feature matching) with deep learning (YOLOv8-nano for object classification, and a neural acoustic model inside Vosk for offline speech recognition), choosing each technique where it is most appropriate for a resource-constrained embedded platform. The system runs with no internet connection and no cloud dependency. We describe the architecture, hardware, software implementation, the engineering challenges encountered while integrating real hardware, and the current results, before outlining future work toward full indoor navigation.

Keywords: Raspberry Pi, assistive technology, edge AI, deep learning, object detection, speech recognition, computer vision, offline navigation.

Contents

1	Project Idea	3
1.1	Motivation	3
1.2	Core Concept	3
1.3	Scope Decision	3
1.4	Related Approaches	3
1.5	Classical Computer Vision vs. Deep Learning in This Project	3
2	System Design & Methodology	4
2.1	Architecture Overview	4
2.2	Awareness Loop	4
2.3	Guided Navigation — Reactive Wall-Following	5
2.4	Voice Destination Input	5
3	Hardware & Software Details	5
3.1	Hardware Components	6
3.2	Software Architecture	6
3.3	AI / Machine Learning Components	7
3.3.1	Why YOLO Inference Is Gated, Not Continuous	7
3.3.2	Why ORB Is Used for Landmarks, Not YOLO	8
3.4	Guided Navigation Decision Flow	8
3.5	Key Configuration Parameters	9
4	Results & Contributions	10
4.1	Current Status	10
4.2	Contributions	10
4.3	Engineering Challenges	11

5	Future Work	11
6	Conclusion	11

1 Project Idea

1.1 Motivation

Visually impaired individuals face daily challenges navigating indoor environments safely and independently, particularly in unfamiliar or changing spaces. Commercial assistive devices that solve this problem comprehensively (e.g. devices using full SLAM-based mapping) are expensive and depend on specialized hardware such as inertial measurement units (IMUs) or LiDAR. Our goal was to build a low-cost, fully offline assistive device using commodity hardware that a student project can realistically build and demonstrate within an academic term.

1.2 Core Concept

The system is shaped like, and worn or carried like, a cane. It listens for a spoken destination, then guides the user there while continuously watching for obstacles and announcing them aloud through Bluetooth headphones. Three questions drive the entire design:

1. **What is in front of me?** — Obstacle detection (depth) and obstacle classification (deep learning).
2. **Where am I?** — Visual place recognition against previously trained locations.
3. **How do I get to my destination?** — Voice-driven guided navigation with turn-by-turn audio.

1.3 Scope Decision

The original project vision considered full indoor mapping using Visual SLAM (e.g. ORB-SLAM3) to build a 3D map and plan routes autonomously. After evaluating the available hardware — no IMU, no wheel odometry, a single Raspberry Pi 4 CPU — we determined that full SLAM was not achievable within the project timeline with acceptable reliability. We therefore redesigned the system around two achievable, complementary modes described in Section 2, while preserving the original vision explicitly as future work (Section 5).

1.4 Related Approaches

Commercial and research assistive navigation devices typically fall into two categories. **Sensor-fusion SLAM systems** (e.g. wearable LiDAR + IMU rigs) build a persistent 3D map and can route to unseen destinations, but require expensive sensors and significant onboard compute, putting them out of reach for a low-cost student prototype. **Simple sensory substitution devices** (e.g. ultrasonic canes that simply beep when close to an obstacle) are cheap and reliable but provide no semantic information — the user is warned *that* something is near, not *what* it is or *where they are*. This project deliberately sits between the two: it adds semantic understanding (object classification, place recognition, voice interaction) on top of low-cost depth sensing, without requiring the full sensor suite SLAM depends on.

1.5 Classical Computer Vision vs. Deep Learning in This Project

A central design decision was choosing, for each recognition task, whether classical computer vision or a learned model was more appropriate given the constraints of a single-board computer with no GPU. Table 1 summarizes this reasoning, which is revisited in more depth in Section 3.3.

Technique	Type	Training data	Used for
ORB + BFMatcher	Classical CV (hand-engineered features)	A 2-minute capture session per location/landmark, no labeling	Place and landmark recognition
YOLOv8-nano	Deep learning (CNN, pretrained)	None (uses public COCO weights)	Classifying generic obstacles (chair, person, etc.)
Vosk acoustic model	Deep learning (DNN-HMM, pre-trained)	None (uses public English model)	Offline speech-to-text for destination input

Table 1: Recognition techniques used in the project, contrasted by type and data requirements.

2 System Design & Methodology

2.1 Architecture Overview

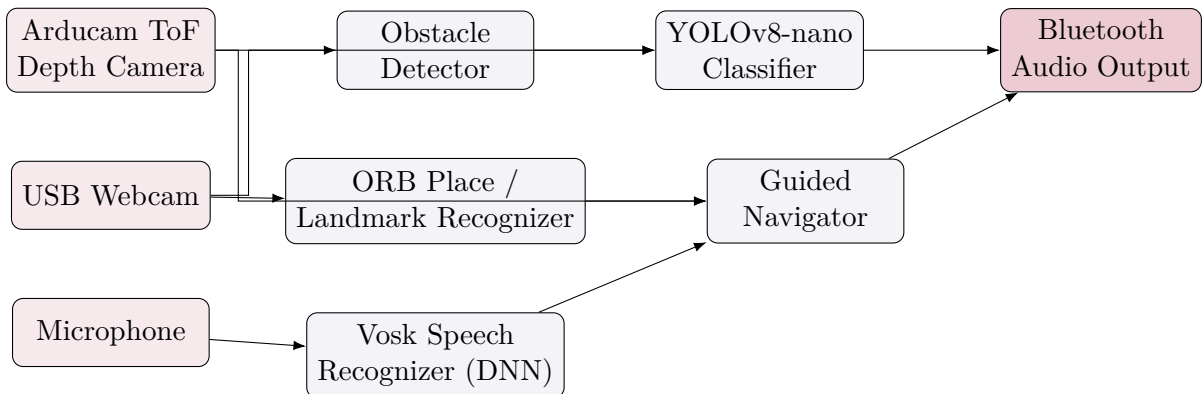


Figure 1: High-level data flow. Sensors (left) feed both classical CV and DL processing stages (center), which converge on the audio output (right).

The system runs two distinct operating modes built from the same underlying modules:

- **Awareness mode** — a continuous 2 Hz loop that detects obstacles and passively announces recognized locations, with no destination.
- **Guided navigation mode** — the user speaks a destination; the system gives turn-by-turn audio guidance, announces landmarks (e.g. doors) along the way, and confirms arrival.

2.2 Awareness Loop

Two concerns run every cycle: a cheap ToF depth threshold check (every cycle) and ORB-based place recognition (every third cycle, to reduce CPU load). Table 2 summarizes the loop.

Concern	Method	Frequency
Obstacle detection	Depth threshold, central zone of ToF frame	Every cycle (2 Hz)
Obstacle classification	YOLOv8-nano on webcam frame	Only when an obstacle is flagged
Location recognition	ORB feature matching vs. trained descriptors	Every 3rd cycle (~ 1.5 s)
Audio output	espeak-ng (TTS) via subprocess	On event

Table 2: Awareness loop concerns and their execution frequency.

2.3 Guided Navigation — Reactive Wall-Following

Without an IMU or wheel odometry, the system cannot plan a path to arbitrary coordinates the way SLAM-based navigation does. Instead, guided navigation reacts to the live ToF depth frame every cycle, split into three vertical zones (left, center, right):

- Center zone clear $\rightarrow$ “*Go straight.*”
- Center zone blocked, one side clearer $\rightarrow$ “*Turn left*” / “*Turn right.*”
- Both sides blocked $\rightarrow$ “*Path blocked. Please stop.*”

This is combined with two independently loaded ORB recognizers: a *location* recognizer (checks for arrival at the destination) and a *landmark* recognizer (announces trained features such as doors when they are recognized close ahead). This approach works reliably for a previously walked, fixed route — a deliberate, honest trade-off documented further in Section 5.

2.4 Voice Destination Input

At the start of guided navigation, the system asks “*Where do you want to go?*” and transcribes the spoken response locally using Vosk, an offline speech-to-text engine. The transcribed text is matched against the labels of previously trained locations. No audio leaves the device — this preserves the project’s fully-offline design goal even for voice interaction.

3 Hardware & Software Details

3.1 Hardware Components

Component	Interface	Role
Raspberry Pi 4 Model B (4GB)	—	Central compute; runs all processing locally
Arducam ToF Camera B0410	CSI	Depth sensing for obstacle detection ($\sim 4\text{m}$ range, 240×180 @ 10fps)
Logitech C270 Webcam	USB (UVC)	Location/landmark recognition, YOLO obstacle classification
Bluetooth Headphones	A2DP	Spoken audio output
Bluetooth/USB Microphone	HSP/HFP or USB	Voice destination input

Table 3: Hardware components and their roles. Approximate total cost: €80.



Figure 2: Assembled prototype: Raspberry Pi 4, Arducam ToF camera (CSI), and USB webcam.

3.2 Software Architecture

The codebase is a modular Python package (`nav_assistant`) organized by concern:

Module	Responsibility
sensors/	Hardware abstraction for the webcam and ToF camera, with automatic V4L2 device detection by name
mapping/	Waypoint data model (SQLite + .npz ORB descriptors), distinguishing location vs. landmark kinds
localization/	ORB-based place/landmark recognition (classical CV)
obstacle/	ToF depth threshold detection
perception/	YOLOv8-nano obstacle classification (deep learning)
speech/	Vosk offline speech-to-text for destination input (deep learning)
navigation/	Reactive wall-following guided navigation
audio/	Text-to-speech (espeak-ng) and alert tones

Table 4: Software module structure.

3.3 AI / Machine Learning Components

The project guidelines require an explicit AI (ML/DL) component. Three recognition techniques are used deliberately, each chosen for where it performs best on constrained edge hardware:

1. **ORB feature matching** (classical computer vision, not learned) — used for visual place and landmark recognition. Chosen because it requires no training data beyond a short capture session, runs entirely on CPU, and is deterministic and debuggable.
2. **YOLOv8-nano** (convolutional neural network, deep learning) — a pretrained, COCO-trained object detector (80 classes, ~6MB) used to classify what triggered a ToF obstacle alert (e.g. “chair”, “person”), turning a generic alert into a specific one.
3. **Vosk speech recognition** (deep neural network acoustic model) — used for fully offline voice destination input, with no cloud speech API involved.

3.3.1 Why YOLO Inference Is Gated, Not Continuous

Running a CNN on every frame would be too slow for real-time use on a Raspberry Pi 4 CPU (no GPU). The architecture instead gates the deep learning inference behind the cheap ToF depth threshold: `ObstacleDetector` runs every cycle as before, and YOLOv8-nano only runs on the corresponding webcam frame *after* an obstacle has already been flagged. This keeps the system real-time while still using a genuine deep learning model for recognition, illustrated in Listing 1.

Listing 1: Obstacle classification gated behind the cheap depth check (`perception/object_detector.py`).

```

1 def classify(self, bgr_frame: np.ndarray) -> Optional[DetectionResult
2     """Return the highest-confidence detection in the frame, or None.
3     """
4     if not self.is_available:
5         return None
6     results = self._model.predict(bgr_frame, verbose=False)[0]
7     if results.bboxes is None or len(results.bboxes) == 0:

```



```
8         return None
9
10        best_idx = int(results.bboxes.conf.argmax())
11        confidence = float(results.bboxes.conf[best_idx])
12        if confidence < self._confidence_threshold:
13            return None
14
15        class_id = int(results.bboxes.cls[best_idx])
16        label = self._model.names[class_id]
17        return DetectionResult(label=label, confidence=confidence)
```

3.3.2 Why ORB Is Used for Landmarks, Not YOLO

“Door” is not a class in the COCO dataset that YOLOv8-nano is pretrained on, and fine-tuning a custom YOLO class would require collecting and labeling a training dataset — impractical within the project timeline. Trained ORB landmarks (Listing 2) solve this with a two-minute capture session per landmark and no labeled dataset, at the cost of being less robust to lighting and viewpoint changes than a learned detector.

3.4 Guided Navigation Decision Flow

Figure 3 illustrates the per-cycle decision sequence implemented by `GuidedNavigator.evaluate()`: arrival is always checked first (highest priority, ends the session), then a landmark announcement is composed if relevant, and finally the wall-following turn/straight decision is computed from the three ToF zones.

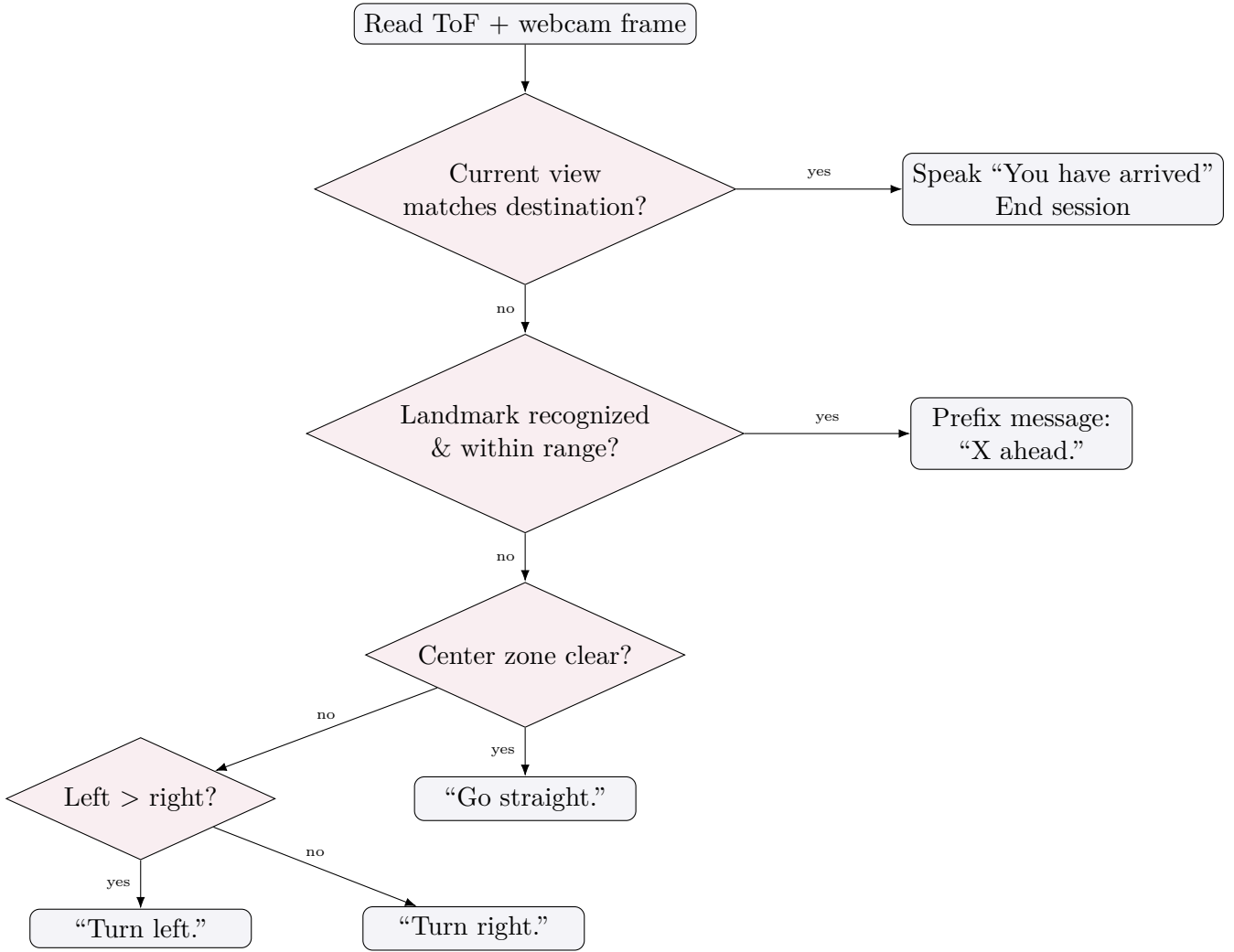


Figure 3: Decision flow for one guided-navigation cycle.

Listing 2: Reactive wall-following decision logic (navigation/guided_navigator.py).

```

1 def _wall_following(self, left: float, center: float, right: float):
2     if center <= 0 or center > self._clear_distance:
3         return GuidanceResult(NavCommand.STRAIGHT, "Go straight.")
4
5     if left <= 0 and right <= 0:
6         return GuidanceResult(NavCommand.STOP, "Path blocked. Please
7             stop.")
8
9     if left > right:
10        return GuidanceResult(NavCommand.TURN_LEFT, "Turn left.")
11    elif right > left:
12        return GuidanceResult(NavCommand.TURN_RIGHT, "Turn right.")
13    else:
14        return GuidanceResult(NavCommand.STOP, "Path blocked. Please
15            stop.")

```

3.5 Key Configuration Parameters

All thresholds are externalized to a single YAML configuration file rather than hardcoded, allowing the system to be re-tuned for a new environment without code changes.

Parameter	Default	Purpose
<code>obstacle.alert_threshold_m</code>	1.2 m	Distance below which an obstacle alert fires (awareness mode)
<code>navigation.clear_distance_m</code>	1.0 m	Center-zone depth above which the path is “clear” (guided navigation)
<code>navigation.emergency_threshold_m</code>	0.4 m	Tight safety override for something dangerously close
<code>navigation.landmark_distance_m</code>	1.5 m	Maximum distance for a landmark to be announced
<code>localization.min_good_matches</code>	15	Minimum ORB matches to accept a recognition
<code>localization.confidence_threshold</code>	0.6	Minimum confidence (0–1) to commit to a recognition
<code>perception.confidence_threshold</code>	0.5	Minimum YOLO detection confidence to report

Table 5: Selected tunable parameters from `config/default.yaml`.

4 Results & Contributions

4.1 Current Status

Phase	Status	Outcome
Sensor integration	Complete	Webcam and ToF camera validated on real hardware with automatic device detection
Location training	Complete	Multiple locations trained, each with 1000–5000 ORB descriptors
Awareness loop	Complete	Real-time obstacle alerts and location announcements running end-to-end
Guided navigation	Implemented	Voice input, wall-following, landmark announcement, arrival detection coded and unit-tested
DL obstacle classification	Implemented	YOLOv8-nano integrated, gated behind ToF threshold

Table 6: Project status by phase.

4.2 Contributions

- A working, fully offline assistive prototype combining depth sensing, computer vision, and two deep learning models on a single Raspberry Pi 4.
- A deliberate, documented architecture that pairs classical CV (ORB) and deep learning (YOLO, Vosk) by matching each technique to where it performs best, rather than defaulting to one approach.
- A robust hardware abstraction layer that automatically detects camera devices by name, solving a real instability (USB device indices changing across reboots) discovered during development.

- 27 automated unit tests covering the data model, sensor processing, navigation decision logic, and DL integration’s graceful degradation.

4.3 Engineering Challenges

Table 7 summarizes the most significant hardware/software integration problems encountered and resolved during development.

Challenge	Resolution
USB webcam device index unstable across reboots	Auto-detect by V4L2 device name at startup instead of a hardcoded index
Arducam SDK API differed from documentation (<code>DeviceType</code> vs. <code>FrameType</code>)	Resolved by reading the official Python examples directly
<code>pyttsx3</code> crashed on Python 3.13 with <code>espeak-ng</code>	Call <code>espeak-ng</code> directly via subprocess, bypassing the broken driver
Single training frame per location was visually fragile	Two-minute continuous capture session, sampling every 3 seconds

Table 7: Key engineering challenges and their solutions.

5 Future Work

The features below were part of the original project vision and were consciously deferred to keep the current scope achievable within the term:

- **Visual SLAM** — building a 3D map of the environment in real time (e.g. ORB-SLAM3) instead of reactive wall-following, enabling navigation to destinations not directly visible.
- **Graph-based route planning** — the data model already stores directed edges between locations (`WaypointEdge`); a Dijkstra-based planner could route across multiple reactive legs.
- **IMU integration** — dead-reckoning between locations would allow distance and orientation tracking independent of visual recognition.
- **Fine-tuned object detection** — training a custom YOLO model on assistive-technology-relevant classes (doors, stairs, curbs) instead of relying on ORB landmarks and generic COCO classes.
- **Multi-environment support** — automatic environment selection when the device is carried between different trained spaces.

6 Conclusion

This project demonstrates that a meaningful assistive navigation system can be built entirely offline on commodity Raspberry Pi hardware by combining classical computer vision and deep learning where each is most effective. Rather than attempting full SLAM-based navigation with insufficient sensors and time, the team redesigned the scope around two achievable, genuinely useful modes — environmental awareness and reactive guided navigation — while explicitly preserving the original ambitions as a documented roadmap. The resulting system detects and classifies obstacles, recognizes trained locations, accepts voice commands, and gives turn-by-turn audio guidance, all without an internet connection.